

Rhynk — Music Provider Architecture

Detailed decision record with implementation sketch · Music Rooms feature · Phase 3 · All-free-tier

1. Decision Overview

Rhynk's music room feature uses a **dual-provider architecture**: JioSaavn as the primary audio source (Spotify-like custom UI) and YouTube's IFrame Player as a fallback (GROIC-style watch-party frame). Both plug into one shared sync engine through a common adapter interface, so switching or adding providers later is a config change, not a rewrite. Every component used is free tier.

Role	Provider	Experience	Cost
Primary	JioSaavn (unofficial API)	Custom audio-only player UI — full brand control	Free
Fallback	YouTube IFrame Player API	Video or audio-only — decided later via CSS, not architecture	Free

2. Primary: JioSaavn

Why

- Matches the actual product pitch — a minimal branded music bar inside chat, not a video player
- Best catalog match for an India-first audience (Hindi/Bollywood/regional languages)
- Full control over play / pause / seek / volume via a custom adapter

Tradeoffs accepted

- No official license — community-maintained wrapper, can change or break without notice
- No SLA, no support channel, possible ToS exposure at real scale
- Acceptable for MVP / beta validation; licensing to be revisited before public launch

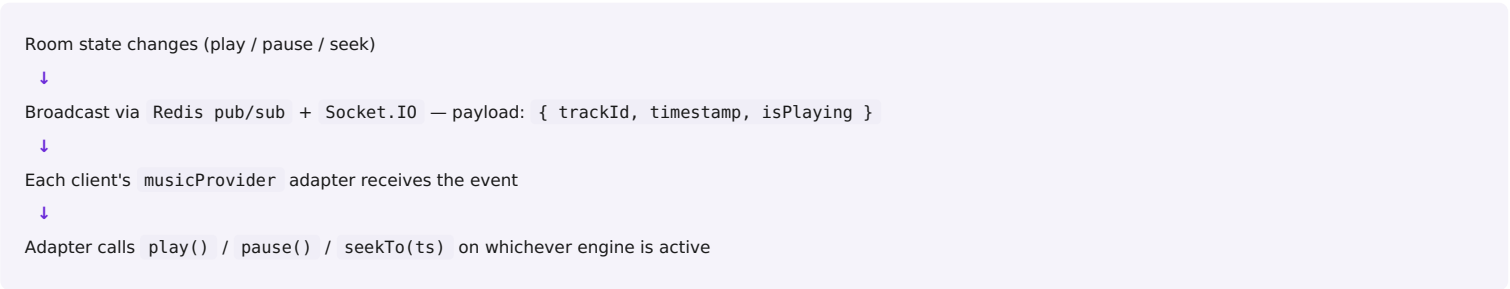
3. Fallback: YouTube IFrame Player (GROIC-style)

- Officially sanctioned embedding — YouTube's own licensing covers playback
- High reliability — won't silently break like an unofficial scraper can
- Confirmed real pattern: GROIC's UI (CC button, cast/PIP icon, settings gear, "Watch on YouTube" pass-through) is the IFrame Player API with a custom skin around it

Video-or-audio-only is a UI decision, deferred:

- With video → show the iframe normally, GROIC-style
- Audio-only → keep the iframe in the DOM but visually hidden; custom audio-bar UI on top (*grey area: YouTube's embed terms expect the video to be viewable, not used purely as an audio backend*)

4. Shared Sync Contract



Both providers expose the same contract to the sync engine, so the hardest engineering problem (room sync, drift correction) is solved **once**, independent of the underlying source.

5. Code Sketch — Adapter Interface

This is the shared contract every provider must implement. It lives on the client, one instance per active room.

CLIENT/SRC/FEATURES/MUSICROOM/PROVIDERS/MUSICPROVIDER.JS

```
// Base contract every provider implementation must follow
class MusicProvider {
  async load(trackId) { throw new Error('not implemented'); }
  play() { throw new Error('not implemented'); }
  pause() { throw new Error('not implemented'); }
  seekTo(timestampSeconds) { throw new Error('not implemented'); }
  getCurrentTime() { throw new Error('not implemented'); }
  onStateChange(callback) { throw new Error('not implemented'); }
}

export function createMusicProvider(type) {
  if (type === 'jiosaavn') return new JioSaavnProvider();
  if (type === 'youtube') return new YouTubeProvider();
  throw new Error(`Unknown provider: ${type}`);
}
```

CLIENT/SRC/FEATURES/MUSICROOM/PROVIDERS/JIOSAAVNPROVIDER.JS

```
export class JioSaavnProvider {
  constructor() {
    this.audio = new Audio();
    this.audio.crossOrigin = 'anonymous';
  }

  async load(trackId) {
    // Backend proxies the unofficial JioSaavn API — never call it from the client
    const res = await fetch(`/api/music/stream/${trackId}`);
    const { streamUrl } = await res.json();
    this.audio.src = streamUrl;
  }

  play() { this.audio.play(); }
  pause() { this.audio.pause(); }
  seekTo(t) { this.audio.currentTime = t; }
  getCurrentTime() { return this.audio.currentTime; }

  onStateChange(cb) {
    this.audio.onplay = () => cb({ isPlaying: true });
    this.audio.onpause = () => cb({ isPlaying: false });
  }
}
```

CLIENT/SRC/FEATURES/MUSICROOM/PROVIDERS/YOUTUBEPROVIDER.JS

```
// Fallback — same contract, wraps the YouTube IFrame Player API
export class YouTubeProvider {
  async load(videoId) {
    this.player = new YT.Player('yt-frame', {
      videoId,
      playerVars: { controls: 0, disablekb: 1 },
    });
  }

  play() { this.player.playVideo(); }
  pause() { this.player.pauseVideo(); }
  seekTo(t) { this.player.seekTo(t, true); }
  getCurrentTime() { return this.player.getCurrentTime(); }

  onStateChange(cb) {
    this.player.addEventListener('onStateChange', (e) => {
      cb({ isPlaying: e.data === YT.PlayerState.PLAYING });
    });
  }
}
```

6. Code Sketch — Sync Broadcast (Server)

Fastify + Socket.IO room handler. Redis holds the current room state so any client joining mid-song gets the correct timestamp immediately.

SERVER/SRC/MODULES/MUSICROOM/SERVICE.JS

```
// Called whenever a user in the room presses play/pause/seek
async function broadcastRoomState(io, redis, roomId, state) {
  // state = { trackId, timestamp, isPlaying, provider }
  await redis.set(`room:${roomId}:state`, JSON.stringify(state));
  io.to(roomId).emit('music:sync', state);
}

// Called when a new client joins an in-progress room
async function getRoomStateForJoin(redis, roomId) {
  const raw = await redis.get(`room:${roomId}:state`);
  if (!raw) return null;

  const state = JSON.parse(raw);
  // adjust timestamp for time elapsed since last broadcast (drift correction)
  const elapsed = (Date.now() - state.updatedAt) / 1000;
  return { ...state, timestamp: state.timestamp + (state.isPlaying ? elapsed : 0) };
}
```

CLIENT/SRC/FEATURES/MUSICROOM/USERROOMSYNC.JS

```
// Client-side: wires socket events to the active provider
function useRoomSync(socket, provider) {
  useEffect(() => {
    socket.on('music:sync', ({ timestamp, isPlaying }) => {
      const drift = Math.abs(provider.getCurrentTime() - timestamp);
      if (drift > 1.5) provider.seekTo(timestamp); // only correct if drift is audible
      isPlaying ? provider.play() : provider.pause();
    });
    return () => socket.off('music:sync');
  }, [provider]);
}
```

7. Fallback Trigger Logic (MVP Scope)

Scenario	MVP Behaviour
JioSaavn track not found / API down at room start	Offer YouTube fallback before the room starts
JioSaavn fails <i>mid-session</i>	Not handled in MVP. Room fails; users reconnect and restart with YouTube fallback. Live mid-room hot-swap deferred to Phase 4 (reliability).

8. Cost Summary — Everything Free Tier

Component	Service	Free-tier limit to watch
Music streaming (primary)	JioSaavn unofficial wrapper	No official quota — but no SLA either; can be rate-limited/blocked without warning
Video/audio fallback	YouTube IFrame Player	No cost for embedding itself
Video search (fallback)	YouTube Data API	Free daily quota — search calls are quota-expensive, cache results
Real-time sync	Upstash Redis (Mumbai)	Existing free-tier account
WebSocket layer	Socket.IO on Fastify	No external cost — self-hosted on existing server

9. Next Step

Wire `createMusicProvider()` into the room's React component so it instantiates `jioSaavnProvider` by default and swaps to `youtubeProvider` only on explicit fallback. Then build the server-side `/api/music/stream/:trackId` proxy route that hides the JioSaavn wrapper behind Rhynek's own API — this is the piece that lets the underlying source change later without touching any client code.